无人机智能巡检系统

AI 智能问答模块

完整技术方案设计文档

文档版本	v1.1（Java 实现版）
编制日期	2026-04-02
文档状态	正式版 — 待评审

一、方案概述

1.1 设计背景

AI 智能问答模块以 DeepSeek 大语言模型为核心，后端使用 Java 17 + Spring Boot 3 实现。为操作员提供自然语言交互入口，覆盖系统全部 16 个业务模块的信息查询与操作引导。本文档描述从前端输入到最终响应的完整技术链路，重点阐述后端的意图识别、工具调用（Tools/API）、信息加工和操作触发四个核心机制的 Java 实现方案。

1.2 整体设计原则

- LLM 负责理解，后端负责执行：意图识别、参数匹配、回答生成均由 DeepSeek 完成；实际 API 调用、数据拉取、权限校验均在 Java 后端执行
- Tools 驱动的函数调用：后端通过 ToolsRegistry Bean 维护业务 API 描述，Spring AI 的 Function Calling 机制让 LLM 自动选择接口并组装参数
- 读写严格分离：查询类意图通过 API 获取数据后由 LLM 加工回答；操作类意图只生成跳转链接和数据模型，不直接调用写接口
- 菜单路由表驱动导航：MenuRegistry Bean 维护菜单操作链接列表，操作触发类意图由 LLM 自动匹配菜单并构建数据模型供前端回填
- Prompt 工程规范化：System Prompt 通过 MCP Server/Skill 机制统一管理和版本化，与业务代码解耦

1.3 核心流程总览

阶段	节点	核心动作
①	AI 问答前端入口	用户输入自然语言问题，前端构造请求体发送至后端
②	AI 问答后端入口	AiChatController 接收请求，组装上下文（历史对话 + 用户问题），路由至意图识别流程
③	意图识别（LLM）	DeepSeek 分析用户意图，分为「普通聊天」和「业务处理」两大分支
④-A	普通聊天	LLM 直接生成回答，无需调用业务 API，直接返回用户
④-B	业务处理	LLM 细分为：信息查询 / 数据统计 / 信息分析 / 业务建议 / 操作触发
⑤	业务数据（业务 API）	ToolsExecutor 按 LLM 选定的 Function 调用对应业务 API，获取实时数据
⑥	信息加工（LLM）	将原始业务数据 + 用户问题交给 LLM 生成规范化自然语言回答
⑦	返回给用户	前端渲染 Markdown 回答 / 操作按钮 / 数据回填表单

二、后端架构设计

2.1 整体分层结构

层次 / 核心类	职责说明
接入层 *AiChatController*	*POST /api/ai/chat* — 统一对话接口；负责请求鉴权、会话 *ID* 绑定、请求体校验、响应格式封装
意图路由层 *IntentRouterService*	调用 *DeepSeek* 第一次 *LLM*（意图识别）；解析返回的 *tool_calls*，路由至 *ChatService* 或 *BusinessService*
Tools 执行层 *ToolsExecutorService*	维护 *ToolsRegistry*；接收 *LLM* 的 *Function Call* 结果；并发调用业务 *API*；聚合多接口数据
操作路由层 *MenuRouterService*	维护 *MenuRegistry*；接收 *LLM* 的菜单匹配结果；构建前端跳转 *ActionResponse*
信息加工层 *AnswerGeneratorService*	调用 *DeepSeek* 第二次 *LLM*（信息加工）；将原始数据转化为自然语言规范化回答
会话管理层 *ConversationService*	持久化对话历史（*Redis/DB*）；维护最近 *5* 轮上下文窗口；会话归档策略

2.2 核心数据结构

2.2.1 ToolDefinition — 业务 API 工具定义

每个业务 *API* 工具对应一个 *ToolDefinition* 对象，注册至 *ToolsRegistry*，由 *Spring* 容器统一管理：

伪代码，提供参考：

```
// ToolsRegistry 单条记录结构
{
  "name": "get_device_list",                    // 工具唯一名称（英文下划线命名）
  "description": "查询无人机设备列表，支持按状态、型号等条件筛选，返回设备的实时状态、
                  电量、位置、飞行时长等信息。适用于：在线设备数量查询、设备状态
                  汇总、低电量设备排查等场景。",  // 详细业务描述（供 LLM 理解）
  "module": "device_management",                // 所属业务模块
  "httpMethod": "GET",                          // HTTP 方法
  "endpoint": "/api/devices",                   // 接口路径
  "parameters": {                               // 入参 JSON Schema
    "type": "object",
    "properties": {
      "status": {
        "type": "string",
        "enum": ["online", "offline", "flying", "maintenance"],
        "description": "设备状态过滤条件，不传则返回全部设备"
      },
      "model": {
        "type": "string",
        "description": "设备型号过滤，如 mavic_3e"
      },
```

```
      "airportId": {
        "type": "string",
        "description": "按归属机场 ID 过滤"
      }
    },
    "required": []
  },
  "responseSchema": {                          // 出参数据结构（供 LLM 理解返回值）
    "type": "array",
    "items": {
      "id": "string",
      "name": "string（设备名称，如 DJI-001）",
      "status": "string（在线/离线/飞行中/维护中）",
      "battery": "number（电量百分比 0-100）",
      "flightHours": "number（累计飞行时长，小时）",
      "location": { "longitude": "number", "latitude": "number" }
    }
  },
  "authRequired": true,                        // 是否需要鉴权
  "sensitiveFields": ["location"]              // 敏感字段（LLM 回答中不直接输出）
}
```

实体类：

```
@Data
@Builder
public class ToolDefinition {

    /** 工具唯一名称（英文下划线，供 LLM Function Calling 使用） */
    private String name;

    /** 详细业务描述（供 LLM 理解适用场景，影响意图匹配准确性） */
    private String description;

    /** 所属业务模块 */
    private String module;

    /** HTTP 方法 */
    private String httpMethod;

    /** 接口路径（相对路径，如 /api/devices） */
    private String endpoint;

    /** 入参 JSON Schema（供 LLM 生成参数，格式符合 OpenAI Function Calling 规范） */
    private Map<String, Object> parametersSchema;

    /** 出参字段描述（供 LLM 理解返回值语义，用于信息加工阶段） */
    private Map<String, Object> responseSchema;

    /** 是否需要用户权限校验 */
    private boolean authRequired;

    /** 敏感字段列表（传入 LLM 前过滤，不输出到回答中） */
    private List<String> sensitiveFields;
}
```

2.2.2 MenuDefinition — 菜单操作定义

每个可操作菜单项对应一个 *MenuDefinition* 对象，注册至 *MenuRegistry*：

伪代码，提供参考：

```
// MenuRegistry 单条记录结构
{
  "menuId": "workorder_create",                  // 菜单唯一标识
  "menuName": "创建工单",                         // 菜单中文名称（供 LLM 匹配）
  "description": "跳转到工单创建页面，适用于需要为设备故障、异常事件、
                  例行维护等创建维修/排查/检查/保养工单的场景。",
  "pageRoute": "workorders",                     // 前端页面路由（data-page 值）
  "action": "create",                            // 操作类型（create/edit/view）
  "triggerKeywords": [                            // 触发关键词（辅助 LLM 匹配，非强
制）
    "创建工单", "新建工单", "报修", "维修申请", "故障处理"
  ],
  "formModel": {                                  // 表单数据模型（字段定义 + 填充说
明）
    "title": {
      "type": "string",
      "description": "工单标题",
      "fillFrom": "从用户描述的问题或设备名称自动生成",
      "required": true
    },
    "type": {
      "type": "string",
      "enum": ["维修", "排查", "检查", "保养", "升级"],
      "description": "工单类型",
      "fillFrom": "根据用户描述的问题性质判断",
      "required": true
    },
    "priority": {
      "type": "string",
      "enum": ["紧急", "高", "中", "低"],
      "description": "优先级",
      "fillFrom": "根据问题紧急程度和设备重要性判断",
      "required": true
    },
    "deviceId": {
      "type": "string",
      "description": "关联设备 ID",
      "fillFrom": "从用户提到的设备名称提取（如 DJI-003 → 查询设备 ID）",
      "required": false
    },
    "description": {
      "type": "string",
      "description": "问题详细描述",
      "fillFrom": "将用户的问题描述整理为规范的问题说明文字",
      "required": true
    },
    "eventId": {
      "type": "string",
      "description": "关联 AI 事件 ID（如来自事件告警）",
      "fillFrom": "从对话上下文中提取事件编号（如 EVT-2026xxx）",
      "required": false
    }
  },
  "confirmRequired": true,                       // 是否需要用户二次确认
  "permissionRequired": ["operator", "admin"]    // 需要的权限角色
}
```

实体类：

```java
// MenuDefinition.java — 菜单操作定义
package com.uav.ai.menu;

import lombok.Builder;
import lombok.Data;
import java.util.List;
import java.util.Map;

@Data
@Builder
public class MenuDefinition {

    /** 菜单唯一标识（供 LLM trigger_menu 工具匹配） */
    private String menuId;

    /** 菜单中文名称 */
    private String menuName;

    /** 详细描述（描述适用场景，供 LLM 精准匹配） */
    private String description;

    /** 前端页面路由（对应 data-page 属性值） */
    private String pageRoute;

    /** 操作类型：create / edit / view / export */
    private String action;

    /** 触发关键词（辅助匹配，非强制） */
    private List<String> triggerKeywords;

    /**
     * 表单数据模型字段定义
     * Key: 字段名，Value: FormFieldDefinition（含类型、描述、填充说明）
     */
    private Map<String, FormFieldDefinition> formModel;

    /** 是否需要用户二次确认才执行跳转 */
    private boolean confirmRequired;

    /** 需要的权限角色列表 */
    private List<String> permissionRequired;


    @Data
    @Builder
    public static class FormFieldDefinition {
        private String type;           // string / number / boolean / enum
        private List<String> enumValues;
        private String description;    // 字段业务含义
        private String fillFrom;       // LLM 填充说明（从哪里提取字段值）
        private boolean required;
    }
}
```

三、完整处理流程详解

3.1 请求接收与上下文构建

AiChatController 接收请求后，*ConversationService* 完成上下文组装：

伪代码，提供参考：

```
// 发送给 DeepSeek 的完整请求结构（意图识别阶段）
{
  "model": "deepseek-chat",
  "messages": [
    {
      "role": "system",
      "content": "[System Prompt — 见第五章]"
    },
    // ... 最近 5 轮历史对话（role: user/assistant 交替）
    {
      "role": "user",
      "content": "DJI-003 最近有几条维修工单？帮我看看"
    }
  ],
  "tools": [/* ToolsRegistry 全量列表 */],
  "tool_choice": "auto"                        // 让 LLM 自动决定是否调用 Tool
}
```

View 层：

```
@RestController
@RequestMapping("/api/ai")
@RequiredArgsConstructor
public class AiChatController {

    private final IntentRouterService intentRouterService;
    private final ConversationService conversationService;

    @PostMapping("/chat")
    public ChatResponse chat(@RequestBody ChatRequest request,
                             @AuthenticationPrincipal UserPrincipal user) {
        // 1. 会话绑定（无 conversationId 则创建新会话）
        String convId = conversationService.resolveConversation(
                request.getConversationId(), user.getUserId());

        // 2. 加载最近 5 轮历史作为上下文（最多 10 条消息）
        List<Message> context = conversationService.loadContext(convId, 5);

        // 3. 持久化本次用户消息
        conversationService.saveMessage(convId,
                Message.user(request.getMessage()));

        // 4. 路由至意图识别 → 执行 → 返回
        ChatResponse response = intentRouterService.route(
                request.getMessage(), context, user);

        // 5. 持久化 AI 回答
        conversationService.saveMessage(convId,
                Message.assistant(response.getContent()));
```

```
        return response;
    }
}
```

3.2 意图识别（第一次 LLM 调用）

IntentRouterService 将 *messages* 和 *tools* 发送至 *DeepSeek*，依据返回的 *finishReason* 路由处理分支：

伪代码，提供参考：

```
// DeepSeek 返回的 tool_calls 示例
{
  "finish_reason": "tool_calls",
  "message": {
    "role": "assistant",
    "tool_calls": [
      {
        "id": "call_abc123",
        "type": "function",
        "function": {
          "name": "get_workorder_list",          // 匹配到的 Tool 名称
          "arguments": "{"deviceId": "DJI-003", "status": null}"
          // LLM 自动组装的参数（从用户问题提取）
        }
      }
    ]
  }
}
```

业务层：

```
@Service
@RequiredArgsConstructor
public class IntentRouterService {

    private final ChatClient chatClient;
    private final ToolsRegistry toolsRegistry;
    private final ToolsExecutorService toolsExecutor;
    private final AnswerGeneratorService answerGenerator;
    private final MenuRouterService menuRouter;
    private final PromptLoader promptLoader;

    public com.uav.ai.dto.ChatResponse route(
            String userMessage,
            List<Message> context,
            UserPrincipal user) {

        // 组装 messages: System Prompt + 历史上下文 + 当前用户消息
        List<Message> messages = new ArrayList<>();
        messages.add(new SystemMessage(
                promptLoader.load("intent-recognition")));    // 从 MCP/Skill 加
载
        messages.addAll(context);
        messages.add(new UserMessage(userMessage));

        // 第一次 LLM 调用（带 tools，使用 function calling）
        ChatResponse llmResponse = chatClient.prompt()
                .messages(messages)
```

```
                .tools(toolsRegistry.getAllTools())                    //  注 入
ToolsRegistry
                .call()
                .chatResponse();

        String finishReason = llmResponse.getResult()
                .getMetadata().getFinishReason();

        // 路由判断
        if ("stop".equalsIgnoreCase(finishReason)) {
            // 普通聊天 — 直接返回 LLM 回答
            return com.uav.ai.dto.ChatResponse.text(
                    llmResponse.getResult().getOutput().getContent());
        }

        if ("tool_calls".equalsIgnoreCase(finishReason)) {
            List<ToolCall> toolCalls = llmResponse.getResult()
                    .getOutput().getToolCalls();

            // 操作触发：含 trigger_menu
            boolean hasTriggerMenu = toolCalls.stream()
                    .anyMatch(tc -> "trigger_menu".equals(tc.name()));
            if (hasTriggerMenu) {
                return menuRouter.handle(toolCalls, context, user);
            }

            // 业务处理：调用业务 API → 信息加工
            List<ToolResult> toolResults =
                    toolsExecutor.execute(toolCalls, user);
            return answerGenerator.generate(
                    userMessage, messages, toolCalls, toolResults);
        }

        // 兜底
        return com.uav.ai.dto.ChatResponse.error(
                "INTENT_UNKNOWN", "无法理解您的问题，请换一种表达方式");
    }
}
```

3.3 业务 API 调用（ToolsExecutorService）

ToolsExecutorService 将 *LLM* 返回的 *tool_calls* 并发执行，聚合结果并过滤敏感字段：伪代码，提供参考：

```
@Slf4j
@Service
@RequiredArgsConstructor
public class ToolsExecutorService {

    private final ToolsRegistry toolsRegistry;
    private final WebClient businessApiClient;   // 指向内部业务服务
    private final ObjectMapper objectMapper;
    private final SensitiveFieldFilter sensitiveFilter;

    /**
     * 并发执行所有 tool_calls，返回 tool 结果列表（供下一步信息加工使用）
     */
```

```
    public List<ToolResult> execute(List<ToolCall> toolCalls,
                                    UserPrincipal user) {
        List<CompletableFuture<ToolResult>> futures = toolCalls.stream()
                .map(tc -> CompletableFuture.supplyAsync(
                        () -> executeOne(tc, user),
                        Schedulers.boundedElastic().asExecutor()))
                .toList();

        return futures.stream()
                .map(f -> {
                    try { return f.get(); }
                    catch (Exception e) {
                        log.error("Tool execution failed", e);
                        return ToolResult.error(e.getMessage());
                    }
                })
                .toList();
    }

    private ToolResult executeOne(ToolCall toolCall, UserPrincipal user) {
        ToolDefinition tool = toolsRegistry.findByName(toolCall.name())
                .orElseThrow(() -> new ToolNotFoundException(toolCall.name()));

        // 1. 解析并校验 LLM 生成的参数
        Map<String, Object> params = parseAndValidate(
                toolCall.arguments(), tool.getParametersSchema());

        // 2. 注入用户权限信息（业务 API 服务端做行级权限）
        params.put("_userId", user.getUserId());
        params.put("_roleId", user.getRoleId());

        // 3. 调用业务 API
        String responseJson = businessApiClient
                .method(HttpMethod.valueOf(tool.getHttpMethod()))
                .uri(uriBuilder   ->   buildUri(uriBuilder,   tool.getEndpoint(),
params))
                .retrieve()
                .bodyToMono(String.class)
                .block();

        // 4. 过滤敏感字段（经纬度、storageUrl 等）
        Object data = objectMapper.readValue(responseJson, Object.class);
        Object          filtered          =          sensitiveFilter.filter(data,
tool.getSensitiveFields());

        return                                  ToolResult.success(toolCall.id(),
objectMapper.writeValueAsString(filtered));
    }
}
```

3.4 信息加工（AnswerGeneratorService — 第二次 LLM 调用）

将业务 *API* 原始数据追加至 *messages* 后，再次调用 *DeepSeek* 生成规范化自然语言回答：伪代码，提供参考：

```
// 发送给 DeepSeek 的信息加工请求（messages 追加 tool 结果）
{
  "model": "deepseek-chat",
  "messages": [
    { "role": "system", "content": "[System Prompt]" },
```

```
    // ... 历史上下文 ...
    { "role": "user", "content": "DJI-003 最近有几条维修工单？帮我看看" },
    {
      "role": "assistant",
      "tool_calls": [/* 上一步 LLM 返回的 tool_calls */]
    },
    {
      "role": "tool",
      "tool_call_id": "call_abc123",
      "content": "[{"id":"WO-001","title":"设备故障维修","status":"已完成",...},
                   {"id":"WO-002","title":"例行检查","status":"处理中",...}]"
    }
    // 无需额外指令，LLM 自动基于 tool 结果生成回答
  ]
}
```

业务层：

```
@Service
@RequiredArgsConstructor
public class AnswerGeneratorService {

    private final ChatClient chatClient;
    private final PromptLoader promptLoader;

    /**
     * 将 tool 执行结果注入 messages，第二次调用 LLM 生成规范化回答
     */
    public com.uav.ai.dto.ChatResponse generate(
            String userMessage,
            List<Message> previousMessages,   // 含 System Prompt + 上下文 + 用户
消息
            List<ToolCall> toolCalls,
            List<ToolResult> toolResults) {

        // 追加 assistant 消息（含 tool_calls）和 tool 执行结果
        List<Message> messages = new ArrayList<>(previousMessages);

        // 追加 assistant 占位消息（声明调用了哪些 tool）
        messages.add(AssistantMessage.withToolCalls(toolCalls));

        // 追加每个 tool 的执行结果
        toolResults.forEach(result ->
                messages.add(new ToolResponseMessage(
                        result.getToolCallId(), result.getContent())));

        // 第二次 LLM 调用（信息加工，不带 tools）
        String answer = chatClient.prompt()
                .system(promptLoader.load("answer-generation"))     //  信 息 加 工
Prompt
                .messages(messages)
                .call()
                .content();

        // 构建响应（含 dataSource 标注）
        return com.uav.ai.dto.ChatResponse.builder()
                .type("text")
                .content(answer)
                .intentType(resolveIntentType(toolCalls))
                .dataSource(buildDataSource(toolCalls, toolResults))
```

```
                .relatedResources(extractRelatedResources(answer))
                .build();
    }
}
```

信息加工阶段 *LLM* 的生成要求（通过 *System Prompt* 约束）：

- 数值加粗高亮：关键数字用 *Markdown* **加粗** 标注，便于前端渲染
- 数据来源标注：回答末尾标注数据来自哪个模块，增强可信度
- 脱敏处理：依据 *System Prompt* 中的禁止输出字段列表，不输出坐标、*URL* 等敏感信息
- 长度控制：单次回答不超过 *500* 字，复杂分析可分段展示
- 操作引导衔接：若数据中有异常（如低电量设备），主动建议后续操作并生成 *relatedResources*

3.5 操作触发完整流程（MenuRouterService）

当 *LLM* 识别用户意图为「操作触发」时，走独立的操作路由链路，流程如下：

1. *LLM* 调用专用 *trigger_menu* 工具，返回匹配的 *menuId* 和从对话中提取的参数
2. 后端从 *MenuRegistry* 查找对应菜单配置，获取 *formModel* 字段定义
3. 后端将 *formModel* 字段定义和用户对话上下文再次提交给 *LLM*，请求 *LLM* 填充表单字段
4. *LLM* 依据对话上下文自动推断并填充每个字段值（无法推断的字段留空）
5. 后端构建完整的 *ActionResponse*，包含 *pageRoute*、*formData*、*confirmRequired* 等信息
6. 前端接收 *ActionResponse*，调用 *navigateToPage()* 跳转目标页，同时将 *formData* 注入表单

当 *LLM* 调用 *trigger_menu* 工具时，*MenuRouterService* 负责菜单匹配、设备 *ID* 解析和表单自动填充：

定义：

```
// trigger_menu 专用工具定义（固定注册在 ToolsRegistry 中）
{
  "name": "trigger_menu",
  "description": "当用户意图是执行某项操作（如创建工单、新建任务、生成报告等），
                  调用此工具触发前端菜单跳转。",
  "parameters": {
    "type": "object",
    "properties": {
      "menuId": {
        "type": "string",
        "description": "目标菜单 ID，从 MenuRegistry 中选择最匹配的"
      },
      "reason": {
        "type": "string",
        "description": "选择此菜单的理由（用于日志和用户提示）"
      },
      "rawContext": {
```

```
        "type": "string",
        "description": "从对话中提取的原始语义信息，用于后续表单填充"
      }
    },
    "required": ["menuId", "reason"]
  }
}
```

响应：

```
// 操作触发最终响应结构（ActionResponse）
{
  "type": "action",                              // 响应类型：action（区别于普通文本
回答）
  "content": "已为您准备好工单创建页面，以下信息已自动填充，请确认后提交。",
  "action": {
    "pageRoute": "workorders",                  // 前端跳转的页面路由
    "actionType": "create",                     // 操作类型
    "menuName": "创建工单",
    "confirmRequired": true,
    "formData": {                               // LLM 自动填充的表单数据
      "title": "DJI-003 设备离线故障排查",
      "type": "排查",
      "priority": "高",
      "deviceId": "dji-003-uuid-xxxx",          // 已通过设备名查询转换为 ID
      "description": "设备 DJI-003 连续离线超过 2 小时，需安排技术人员现场排查通信模块。
",
      "eventId": null
    },
    "emptyFields": ["eventId"],                    // 未能自动填充的字段（前端高亮提
示）
    "confirmMessage": "以上信息由 AI 自动填充，请核对后点击「提交工单」。"
  },
  "dataSource": [{ "module": "设备管理", "api": "GET /api/devices" }]
}
```

业务层

```
@Service
@RequiredArgsConstructor
public class MenuRouterService {

    private final MenuRegistry menuRegistry;
    private final ChatClient chatClient;
    private final PromptLoader promptLoader;
    private final DeviceService deviceService;   // 用于设备名 → ID 转换

    public com.uav.ai.dto.ChatResponse handle(
            List<ToolCall> toolCalls,
            List<Message> context,
            UserPrincipal user) {

        // 1. 提取 trigger_menu 的参数
        ToolCall triggerCall = toolCalls.stream()
                .filter(tc -> "trigger_menu".equals(tc.name()))
                .findFirst().orElseThrow();

        TriggerMenuArgs args = parseArgs(triggerCall.arguments());
```

```
        String menuId = args.getMenuId();
        String rawContext = args.getRawContext();

        // 2. 从 MenuRegistry 查找目标菜单
        MenuDefinition menu = menuRegistry.findById(menuId)
                .orElseThrow(() -> new MenuNotFoundException(menuId));

        // 3. 权限校验
        if (!user.hasAnyRole(menu.getPermissionRequired())) {
            return com.uav.ai.dto.ChatResponse.error(
                    "PERMISSION_DENIED", "您没有执行此操作的权限");
        }

        // 4. 第二次 LLM 调用：表单字段自动填充
        Map<String, Object> formData = fillForm(
                menu.getFormModel(), rawContext, context, user);

        // 5. 构建 ActionResponse
        ActionResponse action = ActionResponse.builder()
                .pageRoute(menu.getPageRoute())
                .actionType(menu.getAction())
                .menuName(menu.getMenuName())
                .confirmRequired(menu.isConfirmRequired())
                .formData(formData)
                .emptyFields(findEmptyFields(formData, menu.getFormModel()))
                .confirmMessage("以上信息由 AI 自动填充，请核对后点击提交。")
                .build();

        return com.uav.ai.dto.ChatResponse.builder()
                .type("action")
                .content("已为您准备好「" + menu.getMenuName() + "」页面，以下信息已
自动填充，请确认后提交。")
                .action(action)
                .build();
    }

    /**
     * 调用 LLM 依据对话上下文填充表单字段
     */
    private Map<String, Object> fillForm(
            Map<String, FormFieldDefinition> formModel,
            String rawContext,
            List<Message> context,
            UserPrincipal user) {

        String formModelJson = objectMapper.writeValueAsString(formModel);

        String prompt = String.format(
                "%s\n\n表单字段定义：\n%s\n\n用户原始描述：%s\n\n" +
                "请按照字段定义从上下文中提取填充值，无法确定的字段返回 null。" +
                "以 JSON 格式返回，key 为字段名，value 为填充值。",
                promptLoader.load("form-filling"),
                formModelJson, rawContext);

        String jsonResult = chatClient.prompt()
                .messages(context)
                .user(prompt)
                .call()
                .content();

        Map<String, Object> formData = objectMapper.readValue(
                jsonResult, new TypeReference<>() {});
```

```
        // 设备名 → ID 转换（如 "DJI-003" → UUID）
        if    (formData.containsKey("deviceId")    &&    formData.get("deviceId")
instanceof String name) {
            formData.put("deviceId",
                    deviceService.findIdByName(name).orElse(null));
        }
        return formData;
    }
}
```

四、ToolsRegistry 与 MenuRegistry 注册实现

4.1 ToolsRegistry — Spring Bean 注册

ToolsRegistry 以 *Spring Bean* 形式注册全部 *24* 个工具，启动时加载，每次 *LLM* 请求注入：

#	工具名称（name）	所属模块	接口	业务描述摘要
1	get_device_list	设备管理	GET /api/devices	查询设备列表，支持按状态/型号/机场筛选
2	get_device_detail	设备管理	GET /api/devices/{id}	获取单台设备详情及型号技术参数
3	get_airport_list	机场管理	GET /api/airports	查询机场列表、运行状态、停机位情况
4	get_mission_list	任务管理	GET /api/missions	查询任务列表，支持状态/类型/日期筛选
5	get_mission_stats	任务管理	GET /api/missions/stats	获取任务完成率、数量分布等统计汇总
6	get_route_list	航线管理	GET /api/routes	查询航线列表，含距离、时长、航点数等
7	get_flight_records	飞行记录	GET /api/flight-records	查询飞行记录，支持按设备/任务/日期筛选
8	get_flight_stats	飞行记录	GET /api/flight-records/stats	获取总飞行时长、距离、架次等统计
9	get_event_list	AI 事件	GET /api/events	查询 AI 检测事件，支持类型/状态/区域筛选
10	get_event_stats	AI 事件	GET /api/events/stats	获取事件类型分布、趋势等统计数据
11	get_report_list	报告系统	GET /api/reports	查询报告列表，支持按类型/时间筛选
12	get_report_summary	报告系统	GET	获取指定报告的

			/api/reports/{id}/summary	摘要统计数据
13	get_workorder_list	工单管理	GET /api/workorders	查询工单列表，支持状态/优先级/设备筛选
14	get_workorder_stats	工单管理	GET /api/workorders/statistics	获取工单数量、SLA 达标率等统计
15	get_algorithm_list	算法配置	GET /api/algorithms	查询 AI 算法列表、阈值、启用状态
16	get_analytics_overview	统计分析	GET /api/analytics/overview	获取系统整体运营 KPI 指标
17	get_analytics_trends	统计分析	GET /api/analytics/trends	获取指定指标的时间趋势数据
18	get_video_list	视频管理	GET /api/media/videos	查询视频列表，支持按设备/任务/日期筛选
19	get_photo_list	图片管理	GET /api/media/photos	查询图片列表，支持按设备/任务/日期筛选
20	get_geofence_list	电子围栏	GET /api/geofences	查询电子围栏列表，支持按类型/状态筛选
21	get_flyarea_list	可飞区域	GET /api/flyableareas	查询可飞区域列表，支持按状态筛选
22	get_entity_list	实体管理	GET /api/entity	查询目标实体列表，支持按类别/状态/威胁等级筛选
23	get_entity_detail	实体管理	GET /api/entity/{id}	查询单个实体详情及毁伤评估数据
24	trigger_menu	系统操作	—（内部路由）	操作触发专用工具，匹配菜单路由并触发前端跳转

业务层：

```
@Component
public class ToolsRegistry {
```

```
    private final Map<String, ToolDefinition> registry = new LinkedHashMap<>();

    @PostConstruct
    public void init() {
        // ── 设备管理 ──────────────────────────────────────────────────────
        register(ToolDefinition.builder()
                .name("get_device_list")
                .description("查询无人机设备列表，支持按状态/型号/机场筛选，返回设备实时
状态、" +
                             "电量、位置、飞行时长。适用于：在线设备数量查询、低电量设备
排查。")
                .module("device_management")
                .httpMethod("GET")
                .endpoint("/api/devices")
                .parametersSchema(Map.of(
                    "type", "object",
                    "properties", Map.of(
                        "status", Map.of("type","string",
                            "enum",
List.of("online","offline","flying","maintenance"),
                            "description","设备状态过滤，不传则返回全部"),
                        "model",  Map.of("type","string","description","设备型号
"),
                        "airportId", Map.of("type","string","description","归属
机场 ID")
                    ),
                    "required", List.of()
                ))
                .responseSchema(Map.of("type","array","items",Map.of(
                    "id","string","name","string（如 DJI-001）",
                    "status","string（在线/离线/飞行中/维护中）",
                    "battery","number（电量百分比 0-100）",
                    "flightHours","number（累计飞行时长，小时）"
                )))
                .authRequired(true)
                .sensitiveFields(List.of("location","longitude","latitude"))
                .build());

        // ── 工单管理 ──────────────────────────────────────────────────────
        register(ToolDefinition.builder()
                .name("get_workorder_list")
                .description("查询工单列表，支持按状态/优先级/设备 ID 筛选。适用于：" +
                             "待处理工单数量查询、指定设备历史维修记录、SLA 超时工单排
查。")
                .module("workorder_management")
                .httpMethod("GET")
                .endpoint("/api/workorders")
                .parametersSchema(Map.of(
                    "type", "object",
                    "properties", Map.of(
                        "status",   Map.of("type","string",

"enum",List.of("UNASSIGNED","ASSIGNED","IN_PROGRESS",

"PENDING_REVIEW","COMPLETED","CLOSED","CANCELLED"),
                            "description","工单状态"),
                        "priority", Map.of("type","string",
                            "enum",List.of("URGENT","HIGH","MEDIUM","LOW"),
                            "description","优先级"),
                        "deviceId", Map.of("type","string","description","关联设
备 ID")
                    ),
```

```
                    "required", List.of()
                ))
                .responseSchema(Map.of("type","array","items",Map.of(
                    "id","string","title","string","status","string",
                    "priority","string","createdTime","string"
                )))
                .authRequired(true)
                .sensitiveFields(List.of())
                .build());

        // ── 操作触发专用工具 ──────────────────────────────────────────────
        register(ToolDefinition.builder()
                .name("trigger_menu")
                .description("当用户意图是执行某项操作（创建工单/新建任务/生成报告等），"
+
                             "调用此工具触发前端菜单跳转和表单预填充。不要用于数据查询场
景。")
                .module("system")
                .httpMethod("INTERNAL")
                .endpoint("INTERNAL")
                .parametersSchema(Map.of(
                    "type", "object",
                    "properties", Map.of(
                        "menuId",     Map.of("type","string",
                            "description","目标菜单 ID，从 MenuRegistry 中选择最匹
配的"),
                        "reason",     Map.of("type","string",
                            "description","选择此菜单的理由"),
                        "rawContext", Map.of("type","string",
                            "description","从对话提取的原始语义，用于表单填充")
                    ),
                    "required", List.of("menuId","reason")
                ))
                .authRequired(true)
                .sensitiveFields(List.of())
                .build());

        // 其余 21 个工具（get_mission_list / get_event_stats / get_entity_list
等）
        // 按相同结构注册，此处省略
    }

    private void register(ToolDefinition tool) {
        registry.put(tool.getName(), tool);
    }

    public Optional<ToolDefinition> findByName(String name) {
        return Optional.ofNullable(registry.get(name));
    }

    /** 返回 Spring AI 所需的 Tool 列表（转换为 OpenAI Function 格式） */
    public List<Object> getAllTools() {
        return registry.values().stream()
                .map(this::toSpringAiTool)
                .toList();
    }
}
```

4.2 MenuRegistry — Spring Bean 注册

MenuRegistry 以 *Spring Bean* 形式注册全部 *14* 个菜单操作项：

#	菜单路由 ID	目标页面	操作类型	触发场景描述
1		missions	create	用户需要创建新的巡检任务、即时/定时/周期任务
2		workorders	create	用户需要为设备故障、AI 事件、维护等创建工单
3		routes	create	用户需要规划新航线、创建飞行路径
4		reports	create	用户需要生成日报/周报/月报或自定义报告
5		equipment	create	用户需要添加新的无人机设备
6		annotations	create	用户需要新建禁飞区或限高区电子围栏
7		annotations	create	用户需要新建可飞区域或调整飞行许可范围
8		events	view	用户需要查看 AI 事件详情、核查告警
9		monitoring	view	用户需要实时监控当前飞行任务或指定设备
10		multi-camera	view	用户需要切换多机位监控视图
11		data	export	用户需要导出图片素材（跳转至照片库）
12		data	export	用户需要导出视频素材（跳转至视频库）
13		algorithms	edit	用户需要调整 AI 算法参数（需管理员权限）
14		maps	create	用户需要在地图上标注新的目标实体

伪代码，提供参考：

```
// MenuRegistry.java
package com.uav.ai.registry;

import org.springframework.stereotype.Component;
import jakarta.annotation.PostConstruct;
import java.util.*;

@Component
public class MenuRegistry {

    private final Map<String, MenuDefinition> registry = new LinkedHashMap<>();
```

```
    @PostConstruct
    public void init() {
        // ── 创建工单 ──────────────────────────────────────────────────────
        register(MenuDefinition.builder()
                .menuId("workorder_create")
                .menuName("创建工单")
                .description("跳转到工单创建页面。适用于：设备故障报修、AI 事件处理、" +
                             "例行维护、设备离线排查等需要创建工单的场景。")
                .pageRoute("workorders")
                .action("create")
                .triggerKeywords(List.of("创建工单","新建工单","报修","维修申请","故
障处理"))
                .formModel(Map.of(
                    "title", FormFieldDefinition.builder()
                        .type("string").description("工单标题").required(true)
                        .fillFrom("从用户描述的问题或设备名称自动生成标题").build(),
                    "type", FormFieldDefinition.builder()
                        .type("enum").enumValues(List.of("维修","排查","检查","保
养","升级"))
                        .description("工单类型").required(true)
                        .fillFrom("根据用户描述的问题性质判断工单类型").build(),
                    "priority", FormFieldDefinition.builder()
                        .type("enum").enumValues(List.of("紧急","高","中","低"))
                        .description("优先级").required(true)
                        .fillFrom("根据问题紧急程度和设备重要性判断优先级").build(),
                    "deviceId", FormFieldDefinition.builder()
                        .type("string").description("关联设备
ID").required(false)
                        .fillFrom("从用户提到的设备名称提取（如 DJI-003 → 查询设备
UUID）").build(),
                    "description", FormFieldDefinition.builder()
                        .type("string").description("问题详细描述
").required(true)
                        .fillFrom("将用户的问题描述整理为规范的说明文字").build(),
                    "eventId", FormFieldDefinition.builder()
                        .type("string").description("关联AI事件
ID").required(false)
                        .fillFrom("从对话上下文提取事件编号（如 EVT-2026xxx）
").build()
                ))
                .confirmRequired(true)
                .permissionRequired(List.of("operator","admin"))
                .build());

        // ── 创建任务 ──────────────────────────────────────────────────────
        register(MenuDefinition.builder()
                .menuId("mission_create")
                .menuName("创建巡检任务")
                .description("跳转到任务创建页面，适用于需要新建即时/定时/周期巡检任务的
场景。")
                .pageRoute("missions")
                .action("create")
                .triggerKeywords(List.of("创建任务","新建任务","安排巡检","调度飞行
"))
                .formModel(Map.of(
                    "name",       FormFieldDefinition.builder().type("string")
                        .description("任务名称").required(true)
                        .fillFrom("根据巡检目标和时间自动生成任务名称").build(),
                    "type",       FormFieldDefinition.builder().type("enum")
                        .enumValues(List.of("instant","scheduled","periodic"))
                        .description("任务类型（即时/定时/周期）").required(true)
```

```
                        .fillFrom("根据用户描述判断任务类型").build(),
                    "deviceId",   FormFieldDefinition.builder().type("string")
                        .description("分配设备 ID").required(false)
                        .fillFrom("从用户提到的设备名称提取").build(),
                    "routeId",    FormFieldDefinition.builder().type("string")
                        .description("关联航线 ID").required(false)
                        .fillFrom("从用户提到的航线名称查询 ID").build(),
                    "priority",   FormFieldDefinition.builder().type("enum")
                        .enumValues(List.of("高","中","低")).description("优先级
").required(true)
                        .fillFrom("根据任务紧急程度判断").build()
                ))
                .confirmRequired(true)
                .permissionRequired(List.of("operator","admin"))
                .build());

        // 其余 12 个菜单项（report_generate / route_create / event_view 等）按相同
结构注册
    }

    private void register(MenuDefinition menu) {
        registry.put(menu.getMenuId(), menu);
    }

    public Optional<MenuDefinition> findById(String menuId) {
        return Optional.ofNullable(registry.get(menuId));
    }
}
```

五、Prompt 设计规范（MCP Server/Skill）

5.1 PromptLoader — Prompt 热加载

System Prompt 通过 *PromptLoader* 从配置中心（或本地 *classpath*）加载，与业务代码解耦，支持热更新：

Prompt 类型	说明
意图识别 *System Prompt*	在第一次 *LLM* 调用时注入，描述意图分类规则、*Tools* 使用规范、*MenuRegistry* 匹配规则
信息加工 *System Prompt*	在第二次 *LLM* 调用时注入，描述回答格式要求、脱敏规则、数据来源标注规范
表单填充 *System Prompt*	在操作触发的表单填充阶段注入，描述字段填充规则、未知字段留空策略

伪代码，提供参考：

```
/**
 * Prompt 加载器，支持从 classpath resources/prompts/ 或远程配置中心加载。
 * 通过 @Cacheable 缓存避免频繁 IO，通过 @CacheEvict 支持热更新。
 */
@Component
public class PromptLoader {

    private static final String PROMPT_BASE_PATH = "prompts/";

    /**
     * 加载指定名称的 Prompt 文本
     * @param promptName 如 "intent-recognition" / "answer-generation" / "form-
filling"
     */
    @Cacheable(value = "prompts", key = "#promptName")
    public String load(String promptName) {
        try {
            var resource = new ClassPathResource(
                    PROMPT_BASE_PATH + promptName + ".txt");
            return resource.getContentAsString(StandardCharsets.UTF_8);
        } catch (IOException e) {
            throw new PromptLoadException("Prompt 加载失败: " + promptName, e);
        }
    }

    /** 手动刷新指定 Prompt 缓存（配合配置中心推送使用） */
    @CacheEvict(value = "prompts", key = "#promptName")
    public void refresh(String promptName) {
        // 缓存清除后，下次 load() 调用自动重新加载
    }
}
```

5.2 意图识别 Prompt（resources/prompts/intent-recognition.txt）

伪代码，提供参考，详细提示词陈越提供：

```
## 角色定义
你是无人机智能巡检系统的 AI 助手，负责理解操作员的自然语言问题，
识别意图并调用正确的业务接口获取数据。

## 意图分类规则

### 1. 普通聊天（不调用任何 Tool）
- 问候语、感谢、闲聊
- 系统使用方法的通用询问
- 无需实时数据的常识性问题
- 处理方式：直接生成回答，finish_reason 为 stop

### 2. 业务处理（必须调用对应 Tool，finish_reason 为 tool_calls）
- 信息查询：查询单条记录或特定对象的属性
- 数据统计：汇总数量、比率、排名等统计指标
- 信息分析：趋势分析、对比分析、异常识别
- 业务建议：基于数据提供优化建议和推荐
- 操作触发：用户表达需要执行某项操作 → 调用 trigger_menu

## Tools 使用规则
1. 只调用已注册的工具，不自行构造 API 路径
2. 参数严格按照每个工具的 parametersSchema 生成，类型必须匹配
3. 一个问题需要多个数据源时，在同一次响应中返回多个 tool_calls
4. 无法确定的参数设为 null，不猜测或捏造参数值

## 操作触发规则
用户表达创建/新建/修改/跳转/导出等操作意图时，调用 trigger_menu：
- 从 MenuRegistry 中选择最匹配的 menuId
- rawContext 字段提取用户提到的关键信息（设备名、问题描述等）

## 禁止输出（任何情况下均不输出）
- 原始坐标数据（经纬度数组、geometry 字段）
- 文件存储 URL（storageUrl、imageUrl、fileUrl 等）
- 用户密码、Token、密钥等安全凭证
```

5.3 信息加工 Prompt（resources/prompts/answer-generation.txt）

```
## 角色定义
你是无人机智能巡检系统的 AI 助手，负责将业务 API 返回的原始数据
加工为清晰、规范、易读的自然语言回答。

## 回答格式要求
1. 关键数值用 **加粗** 标注（如 **42 个任务**、**85% 完成率**）
2. 数量列举超过 5 条时使用 Markdown 列表格式
3. 回答末尾用小字标注数据来源（「数据来源：任务管理模块」）
4. 单次回答控制在 500 字以内

## 主动建议规则
发现以下情况时，在回答末尾主动给出操作建议：
- 设备电量 < 20%
- 未处理事件超过 10 条
- 工单逾期未处理
- 任务失败率 > 20%
```

```
## 脱敏要求
- GPS 坐标替换为区域描述（如"位于工业园区北侧"）
- 文件 URL 不输出，替换为"已有 N 个文件，可在[模块名]页查看"
- 置信度等算法参数正常输出，不脱敏
```

六、前端交互协议

6.1 统一响应结构

后端对所有对话请求返回统一的响应结构，前端依据 *type* 字段决定渲染方式：

```
// 统一响应结构
{
  "conversationId": "conv_uuid_xxxx",            // 会话 ID
  "messageId": "msg_uuid_xxxx",                  // 本条消息 ID
  "type": "text | action | error",               // 响应类型

  // type = "text"（普通文本回答 / 业务查询回答）
  "content": "当前系统共有 **18 台设备在线**, ...",  // Markdown 格式
  "intentType": "query | statistics | analysis | suggestion | chat",
  "dataSource": [                                // 数据来源（供前端展示来源角标）
    { "module": "设备管理", "api": "GET /api/devices", "dataReturned": "18 条记录
" }
  ],
  "relatedResources": [                          // 关联操作按钮（可选）
    { "type": "navigate", "label": "查看设备列表", "pageRoute": "equipment" },
    { "type": "navigate", "label": "创建维修工单", "pageRoute": "workorders",
"action": "create" }
  ],

  // type = "action"（操作触发回答）
  "content": "已为您准备好工单创建页面...",
  "action": {                                    // 见 3.5 节 ActionResponse 结构
    "pageRoute": "workorders",
    "actionType": "create",
    "formData": { ... },
    "emptyFields": [...],
    "confirmRequired": true,
    "confirmMessage": "..."
  },

  // type = "error"（异常情况）
  "error": {
    "code": "PARTIAL_DATA | TIMEOUT | PERMISSION_DENIED | INTENT_UNKNOWN",
    "message": "部分数据获取失败，结果可能不完整",
    "retryable": true
  }
}
```

伪代码，提供参考：

```
@Data
@Builder
public class ChatResponse {

    /** 响应类型：text（普通回答）/ action（操作触发）/ error（异常） */
    private String type;

    /** 会话 ID */
    private String conversationId;

    /** 本条消息 ID */
```

```
    private String messageId;

    // ── type = "text" 时使用 ──────────────────────────────────────────
    /** Markdown 格式的回答正文 */
    private String content;

    /** 意图类型：query / statistics / analysis / suggestion / chat */
    private String intentType;

    /** 数据来源列表 */
    private List<DataSource> dataSource;

    /** 关联操作按钮（可选） */
    private List<RelatedResource> relatedResources;

    // ── type = "action" 时使用 ────────────────────────────────────────
    /** 操作触发详情 */
    private ActionResponse action;

    // ── type = "error" 时使用 ─────────────────────────────────────────
    /** 错误信息 */
    private ErrorInfo error;


    // ── 静态工厂方法 ──────────────────────────────────────────────────
    public static ChatResponse text(String content) {
        return ChatResponse.builder().type("text").content(content)
                .intentType("chat").build();
    }

    public static ChatResponse error(String code, String message) {
        return ChatResponse.builder().type("error")
                .error(ErrorInfo.builder().code(code).message(message)
                        .retryable(!"PERMISSION_DENIED".equals(code)).build())
                .build();
    }


    @Data @Builder
    public static class DataSource {
        private String module;
        private String api;
        private String dataReturned;
    }

    @Data @Builder
    public static class RelatedResource {
        private String type;       // navigate / create
        private String label;
        private String pageRoute;
        private String action;
    }

    @Data @Builder
    public static class ErrorInfo {
        private String code;
        private String message;
        private boolean retryable;
    }
}
```

```java
// ActionResponse.java — 操作触发响应结构
package com.uav.ai.dto;

import lombok.Builder;
import lombok.Data;
import java.util.List;
import java.util.Map;

@Data
@Builder
public class ActionResponse {

    /** 前端跳转的页面路由（对应 data-page 属性值） */
    private String pageRoute;

    /** 操作类型：create / edit / view / export */
    private String actionType;

    /** 菜单中文名称 */
    private String menuName;

    /** 是否需要用户二次确认 */
    private boolean confirmRequired;

    /** LLM 自动填充的表单数据（key 为字段名，value 为填充值） */
    private Map<String, Object> formData;

    /** 未能自动填充的字段名列表（前端高亮提示） */
    private List<String> emptyFields;

    /** 展示给用户的确认说明文字 */
    private String confirmMessage;
}
```

6.2 前端表单预填充处理逻辑

前端接收 *type="action"* 响应后，将 *formData* 存入 *sessionStorage*，目标页面初始化时读取并回填：

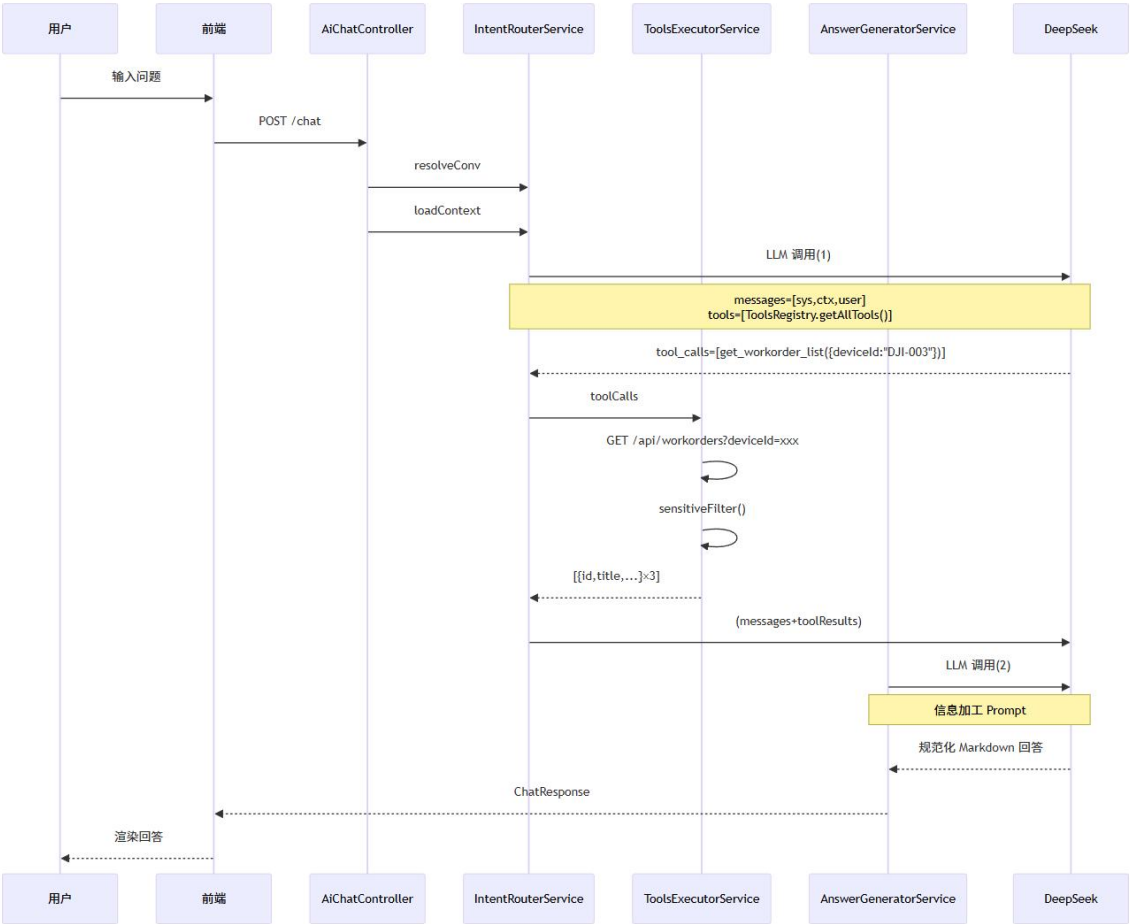
响应类型	触发条件	前端处理
text（普通回答）	*type = "text"*	渲染 *Markdown* 气泡；解析 *relatedResources* 生成操作按钮；底部显示 *dataSource* 来源，*SSE* 流式响应
action（操作触发）	*type = "action"*	渲染确认弹窗；用户确认后 *navigateToPage(pageRoute)*；*formData* 存入 *sessionStorage*；目标页读取后回填表单；高亮 *emptyFields*
error（异常）	*type = "error"*	渲染错误提示；*retryable=true* 时显示重试按钮；*PERMISSION_DENIED* 时显示权限说明

6.3 操作触发前端处理流程

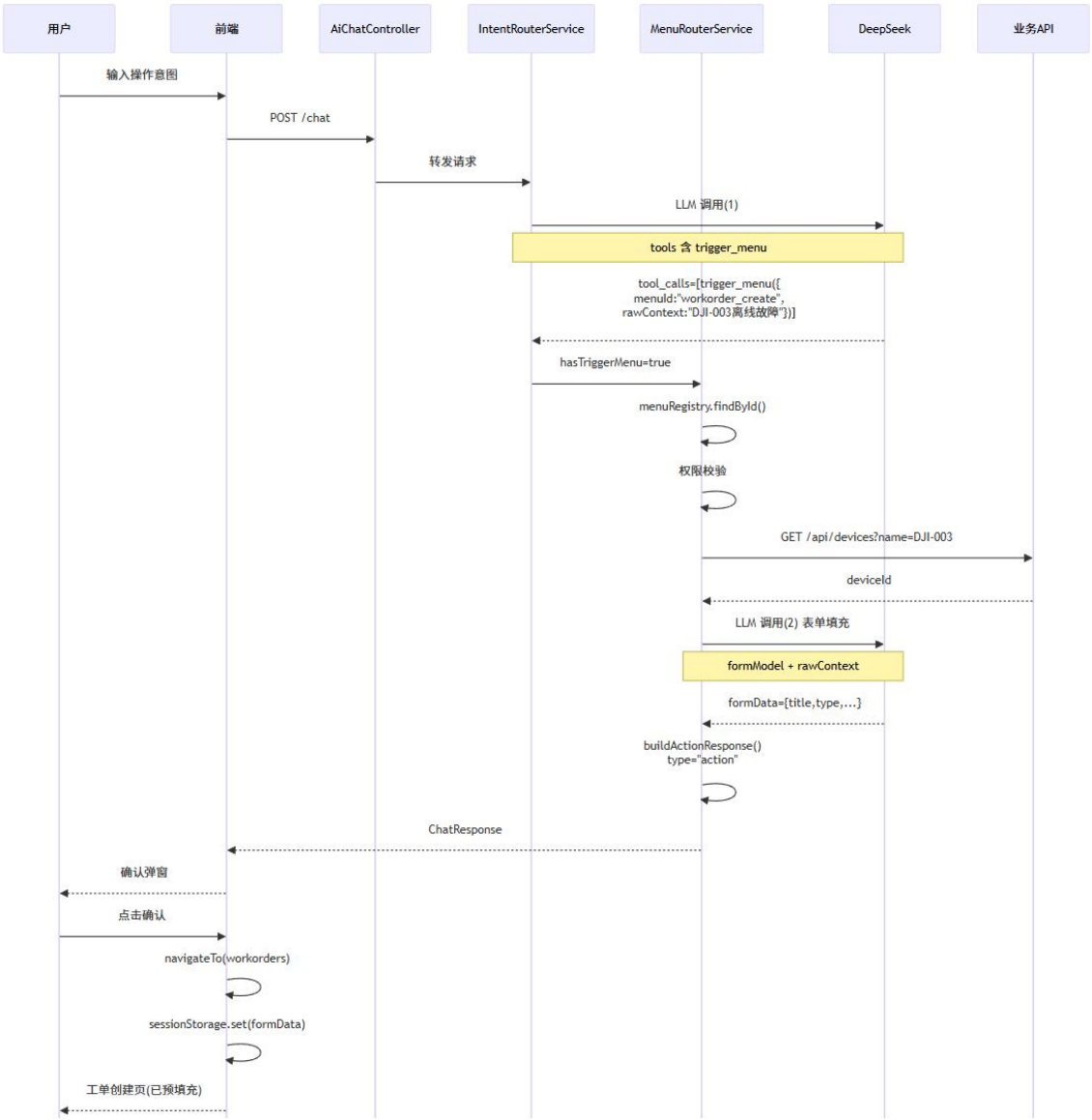
1. 接收 *type = "action"* 响应，渲染 *content* 文本和确认对话框
2. 用户点击「确认跳转」后，调用 *window.navigateToPage(action.pageRoute)*
3. 将 *action.formData* 存入 *sessionStorage*（*key: "ai_prefill_data"*）
4. 目标页面初始化时检测 *sessionStorage* 中的 *ai_prefill_data*，将字段值回填表单
5. 对 *emptyFields* 中列出的字段添加视觉高亮（黄色边框 + 提示"请填写此项"）
6. 表单提交后清除 *sessionStorage* 中的预填充数据

七、完整数据流时序

7.1 业务查询时序（"DJI-003 有几条工单" 示例）



7.2 操作触发时序（"帮 DJI-003 创建工单" 示例）


用户
前端
AiChatController
IntentRouterService
MenuRouterService
DeepSeek
业务API
输入操作意图
POST /chat
转发请求
LLM 调用(1)
tools 含 trigger_menu
tool_calls=[trigger_menu({
menuId:"workorder_create",
rawContext:"DJI-003离线故障"})]
hasTriggerMenu=true
menuRegistry.findById()
权限校验
GET /api/devices?name=DJI-003
deviceId
LLM 调用(2) 表单填充
formModel + rawContext
formData={title,type,...}
buildActionResponse()
type="action"
ChatResponse
确认弹窗
点击确认
navigateTo(workorders)
sessionStorage.set(formData)
工单创建页(已预填充)

八、异常处理与安全设计

8.1 全局异常处理

异常场景	错误码	处理策略
LLM 接口超时（>30s）	TIMEOUT	前端在 5s 无响应时切换提示文案为"分析中，请稍候..."；30s 后中断并提示重试
业务 API 调用失败	PARTIAL_DATA	多 API 并发调用时部分失败不阻断；成功的数据正常返回，失败的在回答中标注"部分数据获取失败"
LLM 参数生成错误	PARAM_ERROR	后端校验 LLM 生成的参数，校验失败时返回 LLM 提示重新生成参数，最多重试 2 次
权限不足	PERMISSION_DENIED	业务 API 返回 403 时，不透传原始错误，向用户说明"无权访问该数据"，不暴露数据结构
意图识别失败	INTENT_UNKNOWN	LLM 无法分类时返回通用提示，展示 3 个快捷问题引导用户重新表达
Tool 不存在	TOOL_NOT_FOUND	LLM 调用了未注册的 Tool 时，后端拦截并返回错误提示，记录日志供 Prompt 优化
LLM 幻觉（输出敏感信息）	CONTENT_FILTER	信息加工层在返回前端前进行关键词过滤（正则匹配坐标、URL 格式），发现后替换为脱敏描述

伪代码，提供参考：

```
@Slf4j
@RestControllerAdvice
public class AiExceptionHandler {

    /** LLM 超时 */
    @ExceptionHandler(LlmTimeoutException.class)
    public ResponseEntity<ChatResponse> handleTimeout(LlmTimeoutException e) {
        log.warn("LLM timeout: {}", e.getMessage());
        return ResponseEntity.ok(ChatResponse.error(
                "TIMEOUT", "AI 分析超时，请稍后重试"));
    }

    /** 工具不存在（LLM 幻觉调用了未注册的 Tool） */
    @ExceptionHandler(ToolNotFoundException.class)
    public ResponseEntity<ChatResponse> handleToolNotFound(ToolNotFoundException e) {
        log.error("Tool not found: {}", e.getToolName());
        return ResponseEntity.ok(ChatResponse.error(
                "TOOL_NOT_FOUND", "服务暂时无法处理该问题，请换一种提问方式"));
    }

    /** 业务 API 部分失败 */
    @ExceptionHandler(PartialApiFailureException.class)
    public ResponseEntity<ChatResponse> handlePartial(PartialApiFailureException e) {
        // 返回已成功获取的数据，标注不完整
```

```
        return ResponseEntity.ok(
                e.getPartialResponse().toBuilder()
                        .content(e.getPartialResponse().getContent() +
                                 "\n\n>   注意：部分数据获取失败，结果可能不完整。")
                        .build());
    }

    /** 权限不足 */
    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ChatResponse> handlePermission(AccessDeniedException e) {
        return ResponseEntity.ok(ChatResponse.error(
                "PERMISSION_DENIED", "您没有权限查看该数据"));
    }

    /** 菜单不存在 */
    @ExceptionHandler(MenuNotFoundException.class)
    public ResponseEntity<ChatResponse> handleMenuNotFound(MenuNotFoundException e) {
        return ResponseEntity.ok(ChatResponse.error(
                "MENU_NOT_FOUND", "未找到对应的操作页面，请手动导航至目标功能"));
    }
}
```

8.2 敏感字段过滤器

```
/**
 * 敏感字段过滤器：在 API 数据传入 LLM 前过滤指定字段，
 * 防止坐标、存储 URL 等敏感信息进入 LLM 上下文并被输出到回答中。
 */
@Component
public class SensitiveFieldFilter {

    private final ObjectMapper objectMapper;

    /** 系统全局敏感字段（无论 Tool 是否声明，均强制过滤） */
    private static final List<String> GLOBAL_SENSITIVE_FIELDS = List.of(
            "longitude", "latitude", "altitude", "location",
            "storageUrl", "imageUrl", "videoUrl", "fileUrl",
            "password", "token", "secretKey", "apiKey"
    );

    public Object filter(Object data, List<String> toolSensitiveFields) {
        JsonNode node = objectMapper.valueToTree(data);
        return filterNode(node,
                mergeLists(GLOBAL_SENSITIVE_FIELDS, toolSensitiveFields));
    }

    private JsonNode filterNode(JsonNode node, List<String> fields) {
        if (node.isArray()) {
            node.forEach(item -> filterNode(item, fields));
        } else if (node.isObject()) {
            ObjectNode obj = (ObjectNode) node;
            fields.forEach(obj::remove);
            obj.fields().forEachRemaining(e -> filterNode(e.getValue(), fields));
        }
        return node;
    }
}
```

8.3 安全设计要点

- 写操作零直连：*AiChatController* 不持有任何业务写接口权限，操作触发只生成前端跳转指令，实际表单提交由前端用户手动完成
- 参数注入防护：*LLM* 生成的参数经 *JSON Schema* 校验（*validateParams*）后才执行，防止 *Prompt Injection* 导致参数篡改
- 用户权限透传：所有业务 *API* 调用均携带 *userId/roleId*，由业务服务端做行级权限控制，*AI* 层不做权限判断
- 敏感字段双重过滤：*Tool* 声明的 *sensitiveFields* + 系统全局 *GLOBAL_SENSITIVE_FIELDS* 在进入 *LLM* 前过滤，*LLM* 回答返回前端前再过滤一次
- 操作确认机制：*confirmRequired = true* 的操作必须经用户显式点击确认才执行跳转，防止误触发
- 会话隔离：*Redis* 缓存 *Key* 包含 *userId* 前缀，不同用户会话严格隔离，*LLM* 上下文不包含他人历史对话
- 审计日志：每次 *LLM* 调用的入参/出参、*Tool* 执行结果、最终回答均异步写入审计日志表（脱敏后），供安全审计和 *Prompt* 优化

九、性能指标与验收标准

9.1 性能目标

指标项	目标值	测试条件	测量方式
第一次 LLM 响应（意图识别）	< 3 秒	标准提问，DeepSeek API 正常	IntentRouterService 计时
业务 API 并发调用	< 1 秒	最多 3 个接口并发	ToolsExecutorService 计时
第二次 LLM 响应（信息加工）	< 4 秒	数据不超过 5KB	AnswerGeneratorService 计时
端到端响应时间（P90）	< 6 秒	业务查询场景	前端计时
操作触发全链路	< 5 秒	含菜单匹配和表单填充	确认弹窗出现时间
并发会话支持	≥ 50 并发	50 用户同时提问	JMeter 压测

9.2 验收检查清单

意图识别准确性

- 给定 20 条测试问题（覆盖 5 类意图），意图识别准确率 ≥ 90%
- 普通聊天类问题不触发任何 Tool 调用（finishReason = stop）
- 操作触发类问题必须调用 trigger_menu，不得直接调用业务写接口

Tool 参数生成质量

- LLM 生成的参数类型符合 parametersSchema，后端 validateParams 无需修正率 ≥ 85%
- 多条件查询（按设备 + 日期过滤）参数正确提取率 ≥ 80%

操作触发回填质量

- 10 条操作触发测试用例，菜单匹配（trigger_menu → menuId）准确率 100%
- 表单字段自动填充准确率（有充分上下文时）≥ 75%
- 确认弹窗正确显示，navigateToPage 跳转后 sessionStorage 中 formData 完整

安全合规

- 任意 100 条测试回答中，无坐标数据、storageUrl、密钥等敏感信息泄露
- 操作触发场景均不直接调用写接口，所有写操作由用户手动提交完成

- 不同用户会话数据严格隔离，*Redis Key* 前缀校验通过

— 文档终 —